



University
Mohammed VI
Polytechnic



Technical Report

Library Microservices System A Cloud-Native Backend Architecture

Mid-Term Assessment – Individual Project

Full name : MIHRAJE Hiba

Program / Year : Second year in DUT IADT at UM6P

Instructor name : EL IDRISSEI Nouhaila

Academic year (2025–2026)

Submission Date: March 2, 2026

Table of content

1. Introduction & System Motivation

2. Architecture Design

3. Primary Business Flow & Inter-Service Communication

4. Cloud-Native Implementation Decisions

5. Deployment & Production Readiness

6. AI Usage Declaration

Introduction & System Motivation

This project focuses on the design and implementation of a [cloud-native backend system for a Library Management service](#). The system is responsible for managing [books, members, and loans](#) using a structured [microservices architecture](#). The main objective of the project is to handle the loan creation process in a controlled and reliable way. This means validating that a member is eligible to borrow a book and confirming that the book is available before creating the loan.

The Library domain was chosen because it naturally allows clear separation between different responsibilities. [Books, members, and loans](#) represent independent entities that can be managed separately. The loan creation process also requires coordinated validation between multiple components, which makes it suitable for demonstrating [inter-service communication and orchestration](#) in a distributed system.

Instead of building [the backend as a single monolithic application](#), the system was intentionally divided into [three independent microservices: Book Service, Member Service, and Loan Service](#). Each service has a well-defined responsibility and communicates with the others using [REST-based HTTP calls](#). The [Loan Service](#) acts as [the orchestrator](#), coordinating validation by calling the Book and Member services when a loan request is received. An [API Gateway](#) is introduced as the single entry point to the system, ensuring that all external requests pass through one controlled interface.

The system follows [cloud-native](#) principles by using [Docker for containerization](#), [Docker Compose for orchestration](#), and [an internal Docker bridge network for secure service-to-service communication](#). Only the [API Gateway](#) is exposed externally, while the internal services remain isolated within [the Docker network](#). Additionally, API documentation is deployed using [Firebase Hosting](#), demonstrating integration with a cloud platform while keeping the backend services protected.

This report explains and justifies the architectural and implementation decisions made throughout the project, including service decomposition, communication patterns, containerization strategy, and cloud integration choices.

Architecture Design

The system is designed using a [microservices architecture](#) composed of three independent services: [Book Service](#), [Member Service](#), and [Loan Service](#). Each service has a specific responsibility and runs in its own [Docker container](#). This separation improves clarity, maintainability, and fault isolation.

The Book Service manages book-related data and provides endpoints to retrieve book information. The Member Service manages member records and validates whether a member exists. The Loan Service is responsible for creating loans and coordinating validation by communicating with both the Book and Member services.

An [API Gateway \(Nginx\)](#) is used as the single public entry point to the system on port **8080**. All external requests pass through the gateway, which routes them to the appropriate internal service. The internal services communicate within a [Docker Bridge Network](#) using private ports (3001, 3002, 3003), ensuring that they are not directly accessible from outside the system.

The [architecture diagram](#) below illustrates the overall structure, network boundaries, request flow, validation logic, and cloud integration.

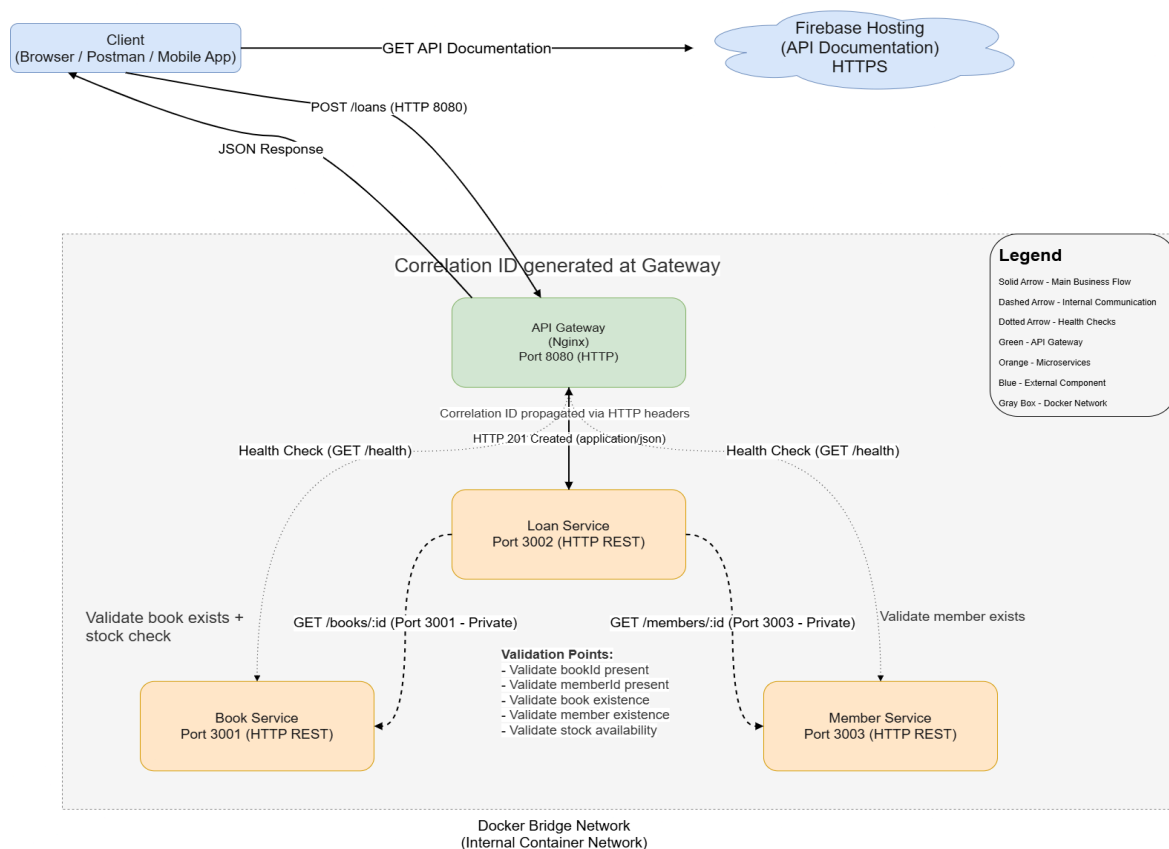


Figure 1: Cloud-Native Library Microservices Architecture

Primary Business Flow & Inter-Service Communication

The primary business operation of the system is the creation of a loan using the endpoint `POST /loans`. When a client sends this request, it first reaches the [API Gateway](#) on port 8080. The gateway then forwards the request to the Loan Service inside the Docker network.

The Loan Service begins by validating the input data. It checks whether the `bookId` and `memberId` are provided. After that, it sends internal `HTTP` requests to the Book Service and the Member Service to verify that the book exists and that the member is valid. The Book Service also checks whether the book is available in stock.

If all validations succeed, the Loan Service creates the loan and returns an `HTTP 201 Created response in JSON format`. This response is sent back through the [API Gateway to the client](#). If any validation fails, the system returns the appropriate error code such as `404 Not Found` or `409 Conflict`.

This flow demonstrates controlled [inter-service communication and clear orchestration](#) within a distributed [microservices environment](#).

Cloud-Native Implementation Decisions

In this project, [Docker](#) was used to run each service separately. Instead of putting everything in one application, each [microservice](#) runs inside its [own container](#). This helps keep the system organized and makes each service independent.

The [API Gateway](#) is the only part of the system that is accessible from outside. All requests from the client go through it. The other services (Book, Member, Loan) are not directly exposed. They communicate with each other inside [the Docker network](#) using their private ports. *This setup makes the system cleaner and more secure because users cannot directly access internal services.*

[Docker Compose](#) is used to start all containers together. With one command, the whole system runs correctly, and all services can communicate with each other. This makes development easier and keeps the structure similar to how modern cloud systems are organized.

Overall, these decisions help keep the system structured, isolated, and easy to manage.

Deployment & Production Readiness

The system is deployed using [Docker Compose](#), which allows all services to run together in an organized way. Each [microservice](#) runs in its own container, which keeps them separated and avoids conflicts.

Only the [API Gateway](#) is exposed to the outside world. All other services stay inside the [Docker network](#) and are not directly accessible. This ensures that every request goes through one controlled entry point.

Each service also includes a [/health](#) endpoint. This makes it possible to check whether a service is running correctly. If one service is not available, [the system returns clear error messages with appropriate HTTP status codes](#).

Even though the system runs locally, the structure is designed in a way that could work in a real production environment. Since each service runs independently, they could be managed or scaled separately if needed.

The API documentation is publicly accessible at:

<https://library-api-docs.web.app>

This deployment demonstrates integration with a real cloud hosting platform while keeping backend services isolated inside Docker containers.

AI Usage Declaration

AI tools (ChatGPT) were used during this project as a support resource for clarification, debugging assistance, and language refinement. Specifically, AI was used to improve grammar in the report, clarify architectural concepts, assist in debugging Docker configuration issues, and guide the restructuring of the lab example from an e-commerce system (Product/Order) into the Library domain (Book/Loan/Member).

All architectural decisions, code modifications, Docker configuration updates, and implementation logic were independently developed and fully understood by me.